FENG CHIA UNIVERSITY · DEPARTMENT OF COMPUTER SCIENCE & INFORMATION ENGINEERING

Software Engineering (SE) — Midterm Examination

Course: Software Engineering (Fall 2026)	Date: Midterm Exam Week	Time Allowed: 100 Minutes	Max Score: 100 Points

Student ID: _____	Student Name: _____	Final Score: _____ / 100

Instructions: Closed book, closed notes. This exam paper contains three sections: **Part I: Multiple Choice (70%)**, **Part II: Fill-in-the-Blank (20%)**, and **Part III: Scenario Essay (10%)**. Write all answers clearly in the designated spaces. For Part I, write the capital letter (A, B, C, or D). For Part III, adhere strictly to the 100–150 word limit.

PART I. MULTIPLE CHOICE QUESTIONS

70 Points (14 Questions · 5 pts each)

1. In Fred Brooks's classic analysis "No Silver Bullet", which challenge represents an *essential difficulty* inherent to the very nature of software rather than an accidental difficulty?

(A) Memory leaks caused by manual pointer management

(B) Slow compilation speed on legacy build toolchains

(C) Conceptual complexity of arbitrary non-linear logic

(D) Syntax errors from unergonomic programming grammar

2. According to Martin Fowler's Technical Debt Quadrant, which scenario exemplifies *Prudent and Deliberate* debt?

(A) Junior developers omitting unit tests because they do not understand test-driven development.

(B) An architecture team intentionally bypassing a microservice boundary to release an MVP before a conference, planning remediation next sprint.

(C) A team copying and pasting legacy code without realizing the severe security vulnerabilities it introduces.

(D) Discovering after three years that a third-party framework is no longer maintained by open-source authors.

3. Under the ISO/IEC 25010 software product quality model, if an application gracefully degrades service and recovers uncommitted database transactions during a network outage, it exhibits high:

(A) Reliability (Fault tolerance & Recoverability)

(B) Usability (Learnability & Operability)

(C) Maintainability (Modularity & Reusability)

(D) Portability (Adaptability & Installability)

4. Under what specific project circumstances is the traditional *Waterfall model* more appropriate and lower risk than an iterative Agile approach?

(A) Consumer mobile apps where UI/UX requirements pivot weekly based on end-user analytics.

(B) Safety-critical avionics firmware with fully validated, unchangeable regulatory specifications.

(C) Early-stage startup prototypes exploring uncharted product-market fit with vague requirements.

(D) E-commerce websites requiring continuous A/B testing and multiple daily production deployments.

PART I. MULTIPLE CHOICE QUESTIONS (CONTINUED) Questions 5 – 9

5. *Brooks's Law* states: "Adding human power to a late software project makes it later." Which mathematical factor primarily explains this counter-intuitive phenomenon?

(A) Hardware CPU throughput degrades linearly when multiple developers write code simultaneously.

(B) Interpersonal communication channels grow combinatorially at $n(n - 1) / 2$, diverting senior developers to training.

(C) Git merge conflict resolution algorithms experience quadratic $O(n^2)$ time complexity in large repositories.

(D) Cloud hosting infrastructure costs increase exponentially whenever extra team members log in.

6. In Lean and Agile Kanban systems, *Little's Law* states $L = \lambda W$ (Lead Time = WIP / Throughput). What is the direct operational consequence of reducing Work-in-Progress (WIP) limits?

(A) Developers work longer overtime hours to compensate for reduced multitasking across feature branches.

(B) Cycle time decreases, bottlenecks become immediately visible, and feedback loops accelerate.

(C) Code quality degrades because developers are strictly forbidden from writing unit tests during WIP caps.

(D) Project scope must be permanently reduced by 50% according to Scrum alliance guidelines.

7. What is the fundamental conceptual difference between *User Requirements* and *System Requirements*?

(A) User requirements specify high-level customer services and operational constraints; System requirements define precise, contractual technical specifications for developers.

(B) User requirements are written strictly in formal mathematical Z-notation; System requirements are written in colloquial natural language.

(C) User requirements are authored exclusively by database administrators; System requirements are written by marketing executives.

(D) User requirements describe only non-functional attributes; System requirements define exclusively GUI wireframes.

8. Which requirement specification statement adheres to best practices by being unambiguous, objectively verifiable, and measurable?

(A) "The order processing subsystem must be user-friendly, responsive, and provide great user satisfaction."

(B) "The checkout API endpoint shall return HTTP 200 within 250 ms at the 99th percentile under a load of 1,000 requests/sec."

(C) "The application should quickly load images whenever high-speed campus Wi-Fi network is available."

(D) "The payment portal shall utilize modern, cutting-edge AI technology to maximize business revenue."

9. In a UML Use Case diagram, when should you model a relationship as `<<extend>>` rather than `<<include>>`?

(A) When the base use case cannot complete its normal flow without unconditionally invoking the target behavior.

(B) When the target behavior represents optional or conditional functionality triggered only at a specific extension point.

(C) When two unrelated human actors share the exact same role and permission levels across the organization.

(D) When one use case inherits all class attributes and database schemas directly from a parent entity.

PART I. MULTIPLE CHOICE QUESTIONS (CONCLUDED) Questions 10 – 14

10. In a UML Class Diagram, what is the key semantic distinction between *Composition* (solid diamond ◆) and *Aggregation* (hollow diamond ◇)?

(A) In Composition, the whole has coincidental ownership, and parts easily outlive the container.

(B) In Composition, parts possess strong lifecycle dependency; when the composite whole is destroyed, its parts are destroyed.

(C) In Aggregation, parts cannot be shared across multiple different parent container objects.

(D) Composition can only be applied to primitive integer data types, whereas Aggregation applies to object pointers.

11. In a UML Sequence Diagram, what visual notation denotes that a participant object is actively executing an internal procedure or waiting for a nested return message?

(A) A dashed vertical timeline called a Lifeline

(B) A vertical rectangular box called an Activation bar

(C) A solid black circle denoting initial state

(D) A horizontal bidirectional dotted arrow

12. In the Boundary-Control-Entity (BCE) architectural pattern, which component is strictly responsible for orchestrating use case business workflows and enforcing transaction rules?

(A) Boundary (View / Web Page / REST API Controller)

(B) Control (Coordinator / Service / Business Logic)

(C) Entity (Data Model / Database Table Schema)

(D) Actor (Human user operating the web browser)

13. In a UML State Machine diagram, under what condition does a transition labeled `submit [isValid == true] / logAudit()` execute?

(A) Whenever event `submit` occurs, regardless of the boolean evaluation of `isValid`.

(B) When event `submit` occurs AND guard condition `isValid == true` holds, subsequently invoking action `logAudit()`.

(C) Only after action `logAudit()` finishes executing and assigns a boolean return value to `submit`.

(D) When the state machine enters an error state and rolls back the database transaction automatically.

14. What is the holistic engineering value of linking Use Case, Sequence, Class, and State diagrams together in a project?

(A) They replace compiler type-checkers so developers no longer need to write unit or regression tests.

(B) They provide complementary structural and behavioral viewpoints, ensuring end-to-end requirement traceability.

(C) They guarantee 100% bug-free software execution without requiring code reviews or CI/CD pipelines.

(D) They allow non-technical project managers to directly generate binary machine code without programmers.

Candidate: _____ (ID: _____)

PART II. FILL-IN-THE-BLANK QUESTIONS

20 Points (10 Questions · 2 pts each)

Instructions: Fill each blank with the foundational term, model, or concept name studied in Chapters 1–4.

15. A visual Lean workflow method restricting WIP using board columns is called _____.

16. A dynamic UML interaction diagram showing messages over lifelines in time order is a _____.

17. In ISO/IEC 25010, how easily users can learn and operate the system is called _____.

18. The classic software model with strictly linear, non-overlapping phases is the _____ model.

19. The implied future cost of rework caused by choosing quick expedient code is called _____.

20. A behavioral UML model capturing external actor goals and interactions is a _____ diagram.

21. The static structural UML diagram modeling classes, attributes, and relationships is the _____ diagram.

22. In UML class models, a strong 'whole-part' association with shared lifecycle is called _____.

23. A UML diagram tracking the event-driven lifecycle states and triggers of an entity is a _____ diagram.

24. Improving internal code structure without changing external behavior is called _____.

PART III. SCENARIO ESSAY QUESTION

10 Points (1 Question · Word Count: 100–150 words)

Question 25 (Scenario & Architectural Trade-off Analysis):
A fast-growing fintech startup rushed an online payment platform to market, declaring: "*We are 100% Agile—working software over comprehensive documentation means writing code directly without UML or layered architecture.*" Three months later, whenever an `Order` is cancelled, orphan `PaymentRecord` rows remain in the DB; the web UI directly executes raw SQL queries; and sprint velocity plummeted by 60% due to cascading regression bugs.
Your Task: Using software engineering principles from Chapters 1–4 (Technical Debt, Agile vs. Cowboy Coding, BCE separation, and UML lifecycle modeling), critically analyze the team's misconception and formulate a concrete **3-step technical remedy** to restore velocity and architectural integrity.
Constraint: Answer must be concise, structured, and strictly between 100 and 150 words.

STUDENT WRITTEN RESPONSE (Target: 100–150 words):

Grading Rubric: [1] Agile vs. Cowboy diagnosis (3 pts) · [2] Architecture & Lifecycle remedy (4 pts) · [3] Strategy & Word limit (3 pts)

Word Count: _____ words

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION ENGINEERING · FCU

Software Engineering Midterm — Official Solution Key

Grading Guide & Conceptual Rationales for Part I (MC) and Part II (Fill-in-the-Blank)

PART I SOLUTIONS: MULTIPLE CHOICE (70 PTS) — 5 Points Each

Q1. Essential Difficulty — **Key: (C)**
Conceptual complexity of arbitrary logic is an inherent essential difficulty (Brooks). Memory, compiler speed, and syntax are accidental tool constraints.

Q2. Technical Debt Quadrant — **Key: (B)**
Intentionally bypassing a boundary for a strategic business deadline while planning immediate payoff is deliberate & prudent (Fowler).

Q3. ISO/IEC 25010 Quality — **Key: (A)**
Graceful degradation and transaction recovery under failure are sub-characteristics of Reliability (Fault Tolerance & Recoverability).

Q4. Waterfall Suitability — **Key: (B)**
Waterfall thrives when specifications are safety-critical, fully validated, and unchangeable upfront. High volatility requires Agile.

Q5. Brooks's Law — **Key: (B)**
Adding personnel increases communication paths combinatorially by $n(n-1)/2$, diverting senior staff into ramp-up coaching.

Q6. Little's Law (Kanban) — **Key: (B)**
$L = \lambda W$. Capping WIP directly reduces cycle time, unmasks system bottlenecks, and enhances feedback velocity.

Q7. User vs. System Reqt — **Key: (A)**
User requirements provide high-level customer services; System requirements specify exact, testable technical behaviors for developers.

Q8. Measurable NFR — **Key: (B)**
Specifying HTTP 200 within 250 ms at 99th percentile under 1,000 req/sec provides a quantitatively testable benchmark.

Q9. Use Case <<extend>> — **Key: (B)**
`<<extend>>` inserts optional, conditional behavior at declared extension points; `<<include>>` is mandatory/unconditional.

Q10. Composition vs. Aggregation — **Key: (B)**
Composition implies strong ownership and coincident lifetime (cascading deletion); Aggregation implies shared, independent existence.

Q11. Sequence Activation — **Key: (B)**
A vertical rectangular bar represents execution focus / active processing period along an object's lifeline.

Q12. BCE Controller — **Key: (B)**
Control objects encapsulate business workflow coordination, state validation, and transactional integrity, insulating UI from Entity.

Q13. State Guard Condition — **Key: (B)**
A transition fires only when the trigger event arrives AND the bracketed guard boolean evaluates to true.

Q14. Holistic Model Linking — **Key: (B)**
Unified diagrams model complementary structural and behavioral facets of a single system, ensuring bidirectional traceability.

PART II SOLUTIONS: FILL-IN-THE-BLANK (20 PTS) — 2 Points Each

15. **Kanban** (or Kanban board)
16. **Sequence Diagram** (or Sequence model)
17. **Usability** (ISO/IEC 25010 characteristic)
18. **Waterfall** (or Waterfall model)
19. **Technical Debt** (Ward Cunningham metaphor)
20. **Use Case** (or Use Case diagram)
21. **Class Diagram** (UML static structural model)
22. **Composition** (Solid diamond ◆ relationship)
23. **State Diagram** (or State Machine diagram)
24. **Refactoring** (Code restructuring technique)

Part III Essay — Grading Rubric & Exemplary Solution

Scoring Criteria, Benchmark Evaluation, and Annotated Model Answer (10 Points)

ESSAY SCORING RUBRIC (TOTAL: 10 POINTS)

Standardized Criteria

Dimension	Mastery / Full Credit	Partial Credit	Inadequate
1. Agile Diagnosis (3 pts)	Exposes that abandoning architecture is *cowboy coding*, not Agile. Connects regression bugs directly to reckless technical debt accumulation and missing validation boundaries.	Mentions technical debt or bad agile, but misses root cause of regression bugs.	Blames programmers or repeats question text.
2. Architecture Remedy (4 pts)	Applies **Composition** (cascading lifecycle deletion to kill orphan records) and **BCE separation** (UI talks to Control service; Control manages Entity transactions).	Suggests database foreign keys or backend API, but lacks UML/BCE design patterns.	Suggests manual DB cleanup scripts.
3. Strategy & Length (3 pts)	Provides structured 3-step remedy (e.g., CI/CD regression suite, WIP debt repayment quota). Strictly satisfies **100–150 word** constraint.	Remedy is vague, or word count is slightly outside 100–150 words (80–99 or 151–175).	Unstructured, or word count violates range severely.

EXEMPLARY MODEL ANSWER (134 WORDS · 10/10 POINTS)

Word Count: 134 words

The team confused Agile with "cowboy coding." Abandoning architectural discipline accumulated reckless technical debt, where regression interest now consumes 60% of velocity.

Concrete 3-Step Remedy:
1. Model Lifecycle Dependencies: Redefine `Order` and `PaymentRecord` as a *Composition* (◆) relationship. Implement cascading deletion and foreign-key integrity constraints to eliminate orphaned records.
2. Enforce Boundary-Control-Entity (BCE): Decouple the UI Boundary from database Entities by inserting a `PaymentControl` service. This coordinator enforces business validation, atomicity, and encapsulates raw SQL into repositories.
3. Pay Down Technical Debt Incrementally: Establish automated CI/CD regression tests and allocate 20% of every sprint capacity to refactor tight coupling, restoring predictable velocity without halting product delivery.

Examiner Notes on Common Pitfalls:

- *Pitfall A:* Suggesting "hire more developers" directly violates Brooks's Law and exacerbates velocity collapse.
- *Pitfall B:* Proposing a complete system rewrite from scratch ignores business continuity; incremental refactoring is the standard engineering practice.
- *Pitfall C:* Writing over 170 words or under 90 words incurs an automatic 1–2 point penalty on conciseness.